

面向资源约束量子布尔函数综合的神经蒙特卡洛树搜索方法

中文论文稿 v36: 图表化投稿主稿

2026 年 7 月 8 日

摘要

量子布尔函数 oracle 综合是 Grover 搜索、约束求解和量子密码分析等算法中的基础编译环节。现有方法通常从固定代数正规形、ESOP/LUT/XAG 逻辑网络、可逆模板或 CNOT-oriented 空间结构出发；这些路线在单一指标或小规模函数上有效，但难以在 T-count、CNOT、深度、门数和辅助比特之间进行统一的资源约束搜索。本文把 bit-flip Boolean oracle synthesis 建模为 ANF/FPRM 项集合上的组合优化问题，并提出 Resource-NMCTS，一种结合神经动作先验、蒙特卡洛树搜索、布尔环线性因子筛选、结构深度策略、frontier policy 与保守 guard 的逻辑层综合框架。本文只讨论逻辑层线路资源，不包含物理设备 mapping、连通性约束、噪声模型或 magic-state factory 调度。

在 $n \leq 6$ 的 177 个传统函数上，Pareto-Resource-NMCTS 相对 direct ANF 平均 T-count 降低 72.25%、平均 score 降低 67.80%；相对 ESOP beam、weighted ESOP-MILP 和 SSHR-H 分别获得 174/0/3、167/3/7 和 173/0/4 的配对胜/负/平。相对 ROS-style LUT proxy，Pareto-Resource-NMCTS 在 309 个函数上为 309/0/0，平均 score 降低 84.27%；相对官方 mockturtle KLUT-to-XAG probe，在传统 177 个函数上为 166/11/0，在 $n = 14$ 的 64 个高维函数上为 64/0/0。内部消融显示，Affine-NMCTS 相对 fixed-coordinate MCTS 平均 score 降低 14.82%，Pareto archive 相对单一 Resource-NMCTS 平均 score 降低 3.26%，learned prior 提供小幅但无 score-loss 的质量增益。

为处理 RevKit `oracle_synth` 的 phase/Rz gate-set 边界，本文实现了可验证的 phase-parity ANF emitter、fixed-polarity FPRM search 和 Affine-FPRM phase search。Affine-FPRM 在 $T/R_z = 30$ proxy 下相对 fixed-polarity FPRM 为 81/0/96、平均 score 降低 2.51%，相对 RevKit 为 177/0/0、平均降低 65.50%。正确性方面，本稿主图汇总的 $n = 20-40$ item-level runs 为 5640/5640，width probe 为 1512/1512，phase selected rows 为 531/531；完整 truth table bridge 在 $n = 21-25$ 上累计 400/400 方法行通过完整 oracle、plan ANF 和 emitted-circuit 符号验证。结果表明，资源约束量子布尔函数综合可以被写成可验证的搜索问题；当前边界是 phase/Rz 分支仍为 parity-gadget 级 proxy，外部 ROS/legacy RevKit-CirKit 仍未完整复现，硬件 mapping 不在本文范围内。

关键词：量子 oracle 综合；布尔函数综合；强化学习；蒙特卡洛树搜索；ANF；FPRM；资源约束编译

1 引言

给定布尔函数 $f : \{0, 1\}^n \rightarrow \{0, 1\}$ ，Boolean oracle synthesis 需要生成实现

$$|x\rangle|y\rangle \mapsto |x\rangle|y \oplus f(x)\rangle \quad (1)$$

的可逆逻辑线路。这个任务看似只是把经典逻辑嵌入量子程序，但在 fault-tolerant 或资源受限量子计算场景中，布尔表示、计算/反计算顺序、辅助比特复用和相位实现方式都会显著改变 T-count、CNOT、深度和辅助比特需求。因此，oracle synthesis 不是一个单步翻译问题，而是一个带有多资源目标和语义约束的离散搜索问题。

已有工作从多个方向处理这个问题。ESOP、XAG、LUT 和 BDD 类方法通过经典逻辑网络降低乘法复杂度或局部函数成本 [1, 2, 3]；RevKit 和 mockturtle 提供了量子/经典逻辑综合工具链入口 [4, 5, 6]；SSHR 则利用 hypercube 中 parallelotope 空间结构进行 small-scale CNOT-oriented synthesis[7]。这些方法给出了重要基线，但它们通常把候选结构或目标指标固定在某一类表示中。对于同时关注 T-count、CNOT、depth、gate count 和 peak ancilla 的逻辑层 oracle 综合，固定表示不一定能覆盖足够的资源折中空间。

本文的出发点是把 oracle synthesis 的主要自由度显式写成搜索空间。我们用 ANF 与 FPRM 项集合表示状态，用布尔环因子、极性选择、线性奇偶因子和可逆仿射变换定义动作，用 MCTS 与神经先验分配搜索预算，并用 guard 保证学习候选不破坏确定性 baseline。这个设计使 AI 不直接生成无约束门序列，而是在可验证的代数动作之间排序。最终线路可以通过 ANF 展开、GF(2) 符号模拟和完整 truth table 逐点验证进行检查。

本文贡献如下：

- 提出 Resource-NMCTS，把量子布尔函数 oracle 综合建模为 ANF/FPRM 项集合上的资源约束搜索问题，而不是 SSHR parallelotope 空间的局部扩展。
- 设计布尔环线性因子筛选、神经动作先验、MCTS、depth-frontier policy、Pareto archive 和 guard 组成的逻辑层 synthesis pipeline。
- 在 ESOP、SSHR、ROS-style LUT proxy、mockturtle XAG probe 和 RevKit phase/Rz proxy 上给出配对对比，并用内部消融拆分搜索、学习和 portfolio 的贡献。
- 用 plan ANF 验证、emitted-circuit GF(2) 符号验证和 $n = 21-25$ 完整 truth table bridge 构建正确性证据链。

a Resource-constrained neural MCTS oracle synthesis remains verifiable at every layer.

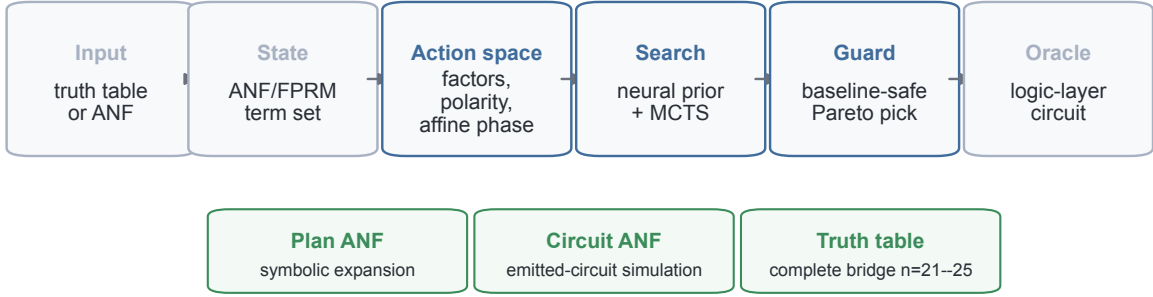


图 1: **Resource-NMCTS** 的可验证搜索 **pipeline**。输入可以是完整 truth table 或 ANF 项集合；动作空间包含布尔环因子、极性、线性奇偶因子和 affine phase 分支；搜索由神经先验、MCTS 和 frontier policy 控制；最终输出在 plan、emitted circuit 和完整 truth table 层进行验证。Source data: fig1_pipeline.csv.

2 相关工作

2.1 布尔表示与 oracle 综合

低 T-count oracle 综合长期依赖合适的布尔中间表示。BDD-based reversible synthesis 以符号决策图处理大函数 [1]；ROS 把 oracle synthesis 写成 resource-constrained LUT mapping，并强调 LUT 分解和后续可逆实现的资源耦合 [2]；XAG 将 XOR 与 AND 结构分离，使 AND 节点数成为非 Clifford 成本的重要 proxy [3]。mockturtle 进一步提供了可复用的 KLUT、XAG 和重综合算法库 [6]。这些路线说明，经典逻辑综合表示会直接影响量子 oracle 资源。

本文与上述工作共享“布尔表示决定量子资源”的前提，但不把搜索限制在某一种固定网络中。我们的状态是 ANF/FPRM 项集合，动作是可展开回 $GF(2)$ 多项式的代数分解。这样做牺牲了部分已有工具链的成熟优化，但换来了更直接的语义验证和更自然的多资源候选组合。

2.2 SSHR 与小规模空间结构方法

SSHR 利用 Boolean hypercube 中 parallelotope 结构生成 MCT block，并以 CNOT reduction 为主要目标 [7]。它适合本文作为 small-scale CNOT-oriented baseline，因为它关注小规模布尔函数、CNOT 计数和空间结构候选。本文不把 SSHR 当作主线方法：Resource-NMCTS 的状态、动作和验证均定义在 ANF/FPRM 项集合和代数搜索上。因此，本文可以在结果中承认 SSHR-H 的 CNOT 优势，同时主张自身在 T-count 和加权 score 上更强。

2.3 学习式搜索与线路综合

学习式搜索已经被用于经典电路设计、量子线路综合和图重写优化。ShortCircuit 使用 AlphaZero 风格搜索处理经典布尔电路设计 [8]；Gumbel AlphaZero 被用于 Clifford+T unitary synthe-

sis[9]; 强化学习也被用于 ZX-calculus rewrite-rule selection[10]; MonteQ 使用 MCTS 处理量子线路合成中的动作排序 [11]。这些工作共同表明, 离散综合问题的动作空间通常过大, 需要学习策略或树搜索分配有限预算。

本文的区别在于, 学习策略不直接写门序列, 而是对可验证的代数动作排序。正确性来自代数展开和 circuit-level symbolic simulation, 而不是来自模型预测本身。这使得 learned prior 的贡献可以通过消融量化; 在当前数据中, 它是小幅正向模块, 而非最大增益来源。

3 问题定义与资源模型

本文考虑 bit-flip Boolean oracle。输入函数可以表示为完整 truth table, 也可以表示为 square-free ANF

$$f(x) = \bigoplus_{m \in \mathcal{M}} \prod_{i \in m} x_i, \quad (2)$$

其中 $\mathcal{M}$ 是 monomial 集合。输出线路使用 X、CNOT 和多控 Toffoli (MCT) 抽象门, 并在逻辑层估计 T-count、CNOT、depth、gate count 与 peak ancilla。

搜索排序采用统一加权分数

$$\text{score} = T + 0.04 \text{ CNOT} + 0.015 \text{ depth} + 0.01 \text{ gates} + 2 \text{ ancilla}. \quad (3)$$

这个分数只用于逻辑层候选排序, 并不代表真实设备运行时间或物理错误率。因此, 本文同时报告 T、CNOT、depth 和 ancilla, 避免单一 score 掩盖 trade-off。

验证分为三层。第一层在小规模函数和高维 bridge 中构造完整 truth table, 逐点验证式 (1)。第二层把 factor plan 展开回 ANF 项集合, 检查目标多项式是否一致。第三层对 emitted X/CNOT/MCT circuit 做 GF(2) 符号模拟, 检查输出线多项式、输入线复原和辅助线清零。第三层不枚举 2^n 个输入, 但验证的是最终 emitted circuit, 而不只是搜索 plan。

4 方法

4.1 项集合状态与可验证动作

Resource-NMCTS 的状态是当前尚未综合的项集合 $\mathcal{M}$ 。一次动作选择一个可计算、可反算的代数结构, 将 $\mathcal{M}$ 分解为 quotient 子问题和 rest 子问题, 再递归生成 compute/action/uncompute 线路。候选动作包括公共单项式因子、FPRM 极性重写后的公共项、线性奇偶因子、布尔环线性因子和 root-action 扩展。动作被接受后, 框架估计即时资源收益、子问题大小、辅助比特峰值和后续可分解性。

神经动作先验用于对候选排序, MCTS 在有限预算内探索候选组合, rollout 使用确定性资源估计和已有 baseline 候选。最终 portfolio 在多个候选线路中按式 (3) 选择最优者。这里 AI 的职责是搜索控制, 而不是语义生成; 任何候选线路都必须通过 ANF 或 circuit-level 验证。

4.2 布尔环线性因子

传统线性因子筛选往往要求线性因子变量和 quotient 变量不相交。本文的 Boolean-ring screen 允许二者共享变量，并在展开时使用 $x_i^2 = x_i$ 与 GF(2) 偶数项消去。例如

$$(x_i \oplus x_j)x_im = x_im \oplus x_ix_jm. \quad (4)$$

线路层面仍可用 CNOT 计算线性奇偶量，再以该辅助量控制 quotient 子线路。该设计扩大了可搜索代数结构，而不是单纯增加 beam width。

4.3 Frontier policy、Pareto archive 与 guard

本文把 AI 模块拆为三类。第一类是动作排序，模型根据候选覆盖率、项数变化、变量覆盖密度、次数分布和即时资源估计对候选动作打分。第二类是结构选择，depth policy 判断当前项集合适合 single、depth-1 还是 depth-2 screen。第三类是 frontier 选择，depth-frontier policy 在 depth-2、depth-3 和 depth-4 screen 之间选择；quality policy 接近 all-depth frontier，cost-aware policy 用小幅度 score 代价换取更低 plan time 和辅助线 lifetime area。

Guard 负责限制学习策略风险。若学习候选相对确定性 baseline 出现 score 退化，则返回 baseline。Pareto archive 在多个候选中保留不同资源折中，再由目标 score 选择最终输出。这个设计使算法可以报告“质量提升”“时间节省”和“质量-时间折中”三类贡献。

5 实验设置

实验回答四个问题。第一，Resource-NMCTS 在完整 truth table 可验证的小规模函数上是否降低 T-count 和 score。第二，与 ESOP、SSHR、ROS-style LUT proxy、mockturtle XAG probe 和 RevKit phase/Rz proxy 相比，结论是否仍成立。第三，内部消融是否能证明提升来自 affine search、MCTS、learned prior、guard 和 Pareto archive，而不是只来自换计分口径。第四，高维项集和 bridge 是否具有足够强的语义验证证据。

传统函数切片包含 $n \leq 6$ 的 177 个函数，baseline 包括 direct ANF、AND-direct ANF、ESOP beam、weighted ESOP-MILP、SSHR-H、ROS-style LUT proxy、mockturtle KLUT-to-XAG probe 和 RevKit Python `oracle_synth`。高维切片包含 $n = 14$ mockturtle probe、 $n = 18, 20$ 函数级边界、 $n = 20-40$ 项集级 scale、width probe，以及 $n = 21, 22, 23, 24, 25$ 完整 truth table bridge。配对比较只纳入函数名或项集名匹配、语义正确且未 timeout 的结果。

6 结果

6.1 传统函数上的多资源优势

图 2 和表 1 给出 $n \leq 6$ 传统函数切片上的平均资源。Pareto-Resource-NMCTS 相对 direct ANF 平均 T-count 降低 72.25%、平均 score 降低 67.80%；相对 weighted ESOP-MILP 平均 T-count 降低 32.77%、score 降低 29.84%。SSHR-H 的平均 CNOT 最低，说明本文不能声称 CNOT-only 支

配 SSHR；更准确的结论是，本文以一定 CNOT 和辅助比特 trade-off 换取更低 T-count、depth 和加权 score。

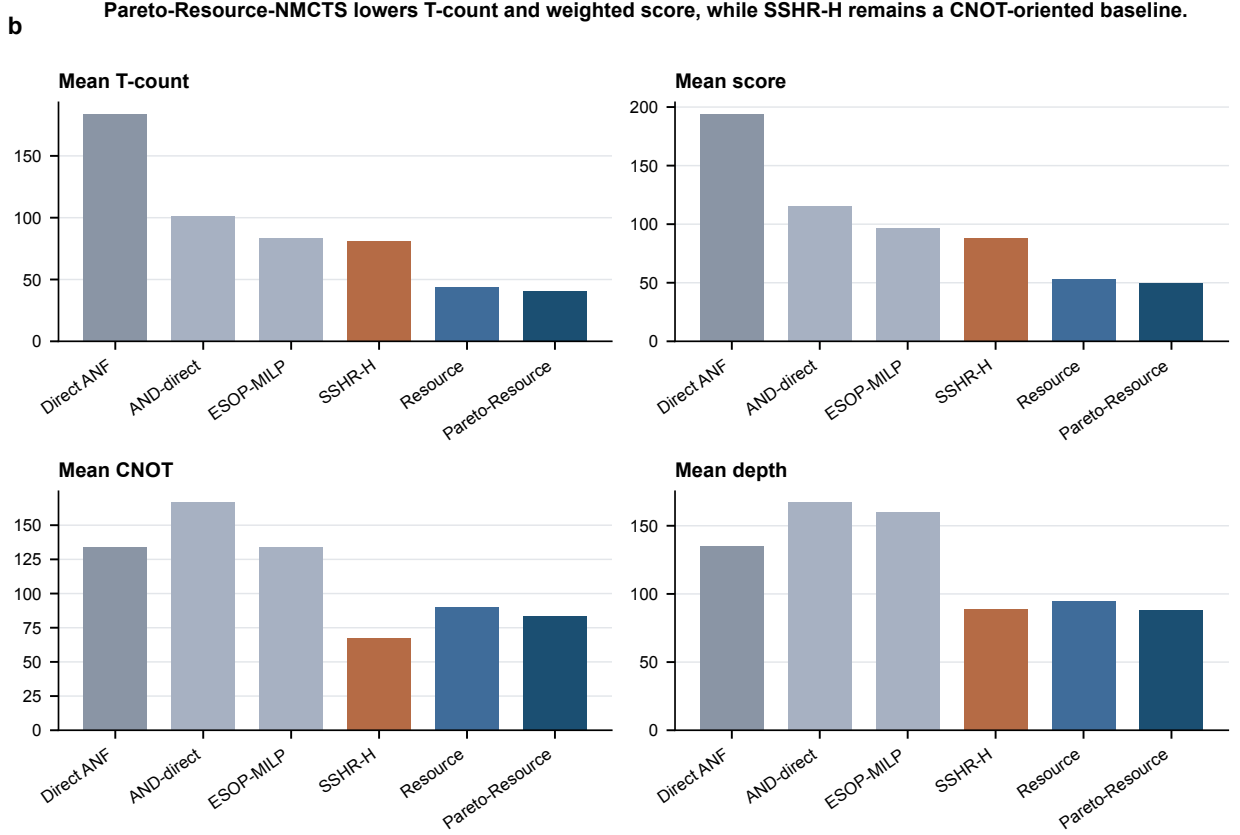


图 2: 传统函数上的平均资源。Pareto-Resource-NMCTS 在 T-count 和 weighted score 上最低；SSHR-H 仍是强 CNOT-oriented baseline。Source data: fig2_traditional_resources.csv.

表 1: $n \leq 6$ 传统函数切片上的平均逻辑资源。

方法	平均 T	平均 CNOT	平均 depth	平均 ancilla	平均 score
Direct ANF	184.1	134.2	134.6	1.33	194.3
AND-direct ANF	101.0	166.5	166.9	2.18	115.2
ESOP-MILP	83.6	133.5	159.6	2.32	96.7
SSHR-H	81.0	67.6	88.7	1.36	88.2
Resource-NMCTS	43.9	89.9	94.5	1.92	53.2
Pareto-Resource-NMCTS	40.4	83.0	88.0	2.03	49.6

6.2 外部 baseline 与可比边界

图 3 汇总外部和传统 baseline 的配对比较。Pareto-Resource-NMCTS 相对 ESOP beam 为 174/0/3, 平均 score 降低 36.09%; 相对 ESOP-MILP 为 167/3/7, 平均 score 降低 29.84%; 相对 SSHR-H 的 T-count 为 173/0/4, 平均降低 47.93%。ROS-style LUT proxy 覆盖 309 个函数, best- K proxy 比固定 $K = 4$ 更强, 但 Pareto-Resource-NMCTS 相对该 proxy 仍为 309/0/0。mockturtle probe 用官方 header 将 KLUT network 重综合为 XAG; 在传统 177 个函数上为 166/11/0, 在

$n = 14$ 的 64 个函数上为 64/0/0。

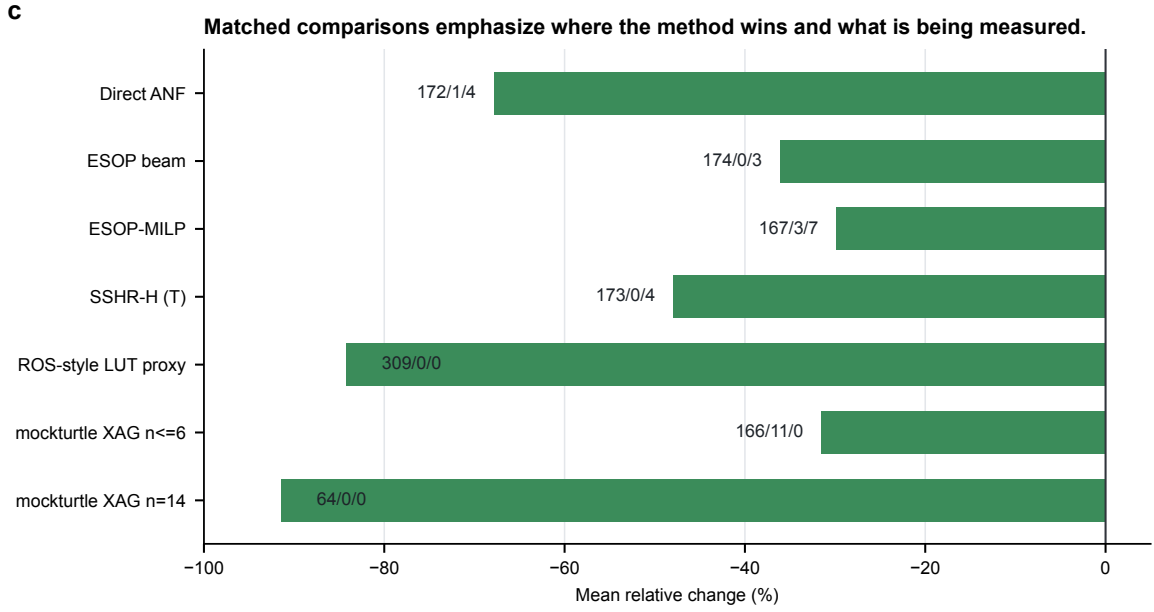


图 3: 配对基线比较。横轴为目标方法相对基线的平均变化；负值表示资源更低。数字标签给出胜/负/平；源数据见 `fig3_baseline_comparisons.csv`。

这些对比有意义,但需要严格限定。ROS-style LUT 是 proxy,不是官方 ROS 全流程;mockturtle probe 是 KLUT-to-XAG 逻辑网络比较,不输出可逆 Clifford+T 线路;RevKit `oracle_synth` 返回 Rz-phase netlist,不能直接当作精确 T/Tdg 网络比较。本文据此只主张逻辑层资源优势,而不是工具链全流程或硬件后端优势。

6.3 内部消融：提升来自哪里

表 2 显示,最大算法增益来自 affine search。Full Affine-NMCTS 相对 fixed-coordinate MCTS 的平均 score 降低 14.82%; Resource-NMCTS 相对 Affine-NMCTS 降低 1.91%; Pareto archive 相对单一 Resource-NMCTS 降低 3.26%。Learned prior 对 Resource-NMCTS 的 score 增益为 1.10%,且无 score-loss,但运行时间上并非最优。因此,本文不把“神经网络”写成决定性单点突破,而把它定位为可验证搜索框架中的小幅正向搜索先验。

表 2: 搜索贡献分解。负的 mean Δ score 表示目标方法更优。

Component	Dataset	Pairs	Score W/L/T	Mean Δ score
Affine greedy vs fixed-coordinate MCTS	ablation_affine	321	165/88/68	-12.12%
Neural refine over affine greedy	ablation_affine	322	65/0/257	-1.08%
Guarded Affine-NMCTS over no-guard	ablation_affine	322	88/0/234	-1.74%
Learned prior for Resource-NMCTS	traditional_resource	177	39/0/138	-1.10%
Pareto archive over Resource-NMCTS	traditional_resource	177	68/0/109	-3.26%
No-MCTS portfolio over heuristic-only	trad. ablation	177	54/0/123	-2.51%
Resource-NMCTS over no-MCTS portfolio	trad. ablation	177	54/0/123	-1.44%
Pareto Resource-NMCTS over no-MCTS portfolio	trad. ablation	177	106/0/71	-4.69%
Highdim no-MCTS portfolio over root beam	n14 ablation	16	14/0/2	-6.25%
Highdim no-MCTS portfolio over linear-pair	n14 ablation	16	14/0/2	-3.08%
Linear-pair guard vs root beam at n=14	highdim_resource	64	60/0/4	-3.00%
Recursive linear-pair guard vs root beam at n=15	n15 scale	32	30/0/2	-5.28%
Recursive linear-pair guard vs shallow linear-pair at n=16	n16 scale	24	23/0/1	-2.54%
Recursive linear-pair guard vs root beam at n=16	n16 scale	24	23/0/1	-4.31%
Baseline-preserving AI guard vs deterministic recursive guard at n=16	n16 scale	24	6/0/18	-0.05%
Fast linear-pair guard vs root beam at n=18	n18 stress	12	6/0/6	-1.91%
Resource-NMCTS vs fast linear-pair at n=18	n18 stress	12	12/0/0	-3.55%

6.4 Phase/Rz 边界与 Affine-FPRM 搜索

RevKit `oracle_synth` 在 177 个传统函数上全部返回，但 171/177 行含非 Clifford R_z ，总数为 9242。因此，如果只按 lower-bound phase netlist 计分，比较会低估 RevKit 输出在 Clifford+T 近似综合中的成本。为把该边界转化为本文内部可验证分支，我们实现 phase-parity ANF emitter，并进一步加入 fixed-polarity FPRM search 和 Affine-FPRM phase search。

图 4 显示，Affine-FPRM 在 $T/R_z = 30$ proxy 下相对 fixed-polarity FPRM 为 81/0/96，平均 score 降低 2.51%；相对 phase-parity ANF 为 85/0/92，平均降低 2.98%；相对 RevKit 为 177/0/0，平均降低 65.50%。在 177 个函数中，81 个选择非 identity 线性变换，43 个选择非零极性。这个结果说明 phase/Rz 分支已经从成本敏感性讨论推进为可验证的代数搜索；但它仍是 parity-gadget 级逻辑 emitter，不包含近似旋转序列生成或硬件映射。

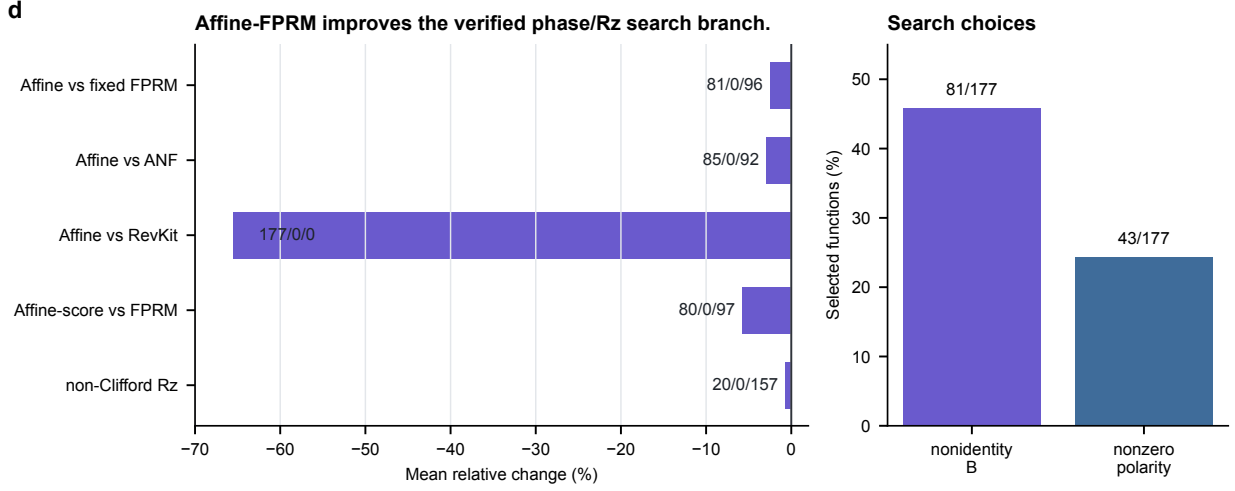


图 4: **Affine-FPRM** 相位搜索。左图为 Affine-FPRM 相对 fixed FPRM、ANF 和 RevKit proxy 的平均变化；右图为非 identity 线性变换和非零极性的选择频率；源数据见 `fig4_phase_affine.csv`。

表 3: Affine-FPRM phase-parity search 的配对比较。

Comparison	Items	W/L/T	Mean Δ
Affine-FPRM- $T/R_z = 30$ vs FPRM	177	81/0/96	-2.51%
Affine-FPRM- $T/R_z = 30$ vs ANF	177	85/0/92	-2.98%
Affine-FPRM- $T/R_z = 30$ vs RevKit	177	177/0/0	-65.50%
Affine-FPRM non-Clifford R_z vs FPRM	177	20/0/157	-0.73%
Affine-FPRM-score vs FPRM	177	80/0/97	-5.77%

6.5 高维验证与 bridge

图 5 汇总本稿主图采用的验证覆盖。 $n = 20$ – 40 item-level runs 为 5640/5640, width probe 为 1512/1512, phase selected rows 为 531/531, 完整 truth table bridge 为 400/400。Bridge 的组成是: $n = 21, 22$ 基础 bridge 120 行, $n = 23$ 基础 bridge 40 行, $n = 23$ large-policy rerun 60 行, $n = 23$ cost-aware rerun 60 行, $n = 24$ 和 $n = 25$ 各 60 行。所有这些方法行同时通过完整 oracle、plan ANF 和 emitted-circuit 符号验证。

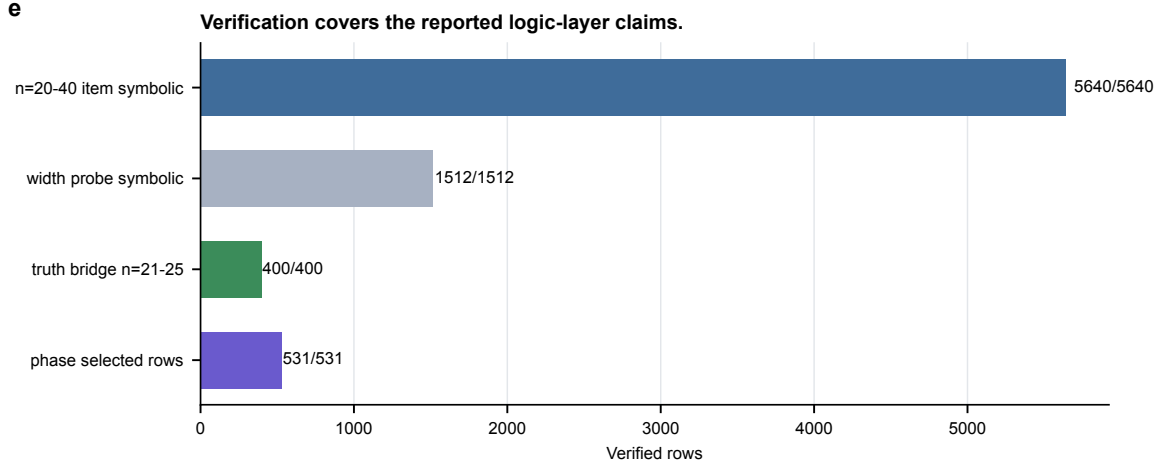


图 5: 验证覆盖。主图汇总的逻辑层结论均有对应 source-data 行；完整 truth table bridge 当前按源数据为 400/400，而不是旧稿中曾使用的 420/420 口径。Source data: fig5_validation.csv.

表 4: 高维验证与边界。

验证项	当前覆盖	边界
$n = 20-40$ item-level symbolic runs	5640/5640	不是完整 truth table 枚举。
Width probe symbolic runs	1512/1512	证明加宽候选不是主收益来源，但仍是项集级验证。
$n = 21-25$ complete bridge	400/400	小样本完整枚举，不能外推为 $n = 26-40$ 全部完整验证。
Phase selected rows	531/531	up to global phase 验证，不输出近似旋转门序列。

7 讨论

本文的核心提升不在于“比 SSHR 多一个启发式”，而在于把 oracle synthesis 改写成资源约束搜索问题。SSHR 在本文中是重要的 CNOT-oriented small-scale baseline；ESOP、ROS-style LUT、mockturtle XAG 和 RevKit phase netlist 则分别从乘积项、LUT、逻辑网络和 phase/Rz gate-set 角度提供外部压力测试。这样设置对比对象是有意义的，因为它们分别覆盖了本文可能被审稿人质疑的四个问题：是否只赢 naive ANF，是否只赢 SSHR，是否能对齐外部工具链，是否处理 RevKit phase/Rz 边界。

同时，当前稿件仍有清晰边界。第一，learned prior 的增益小于 affine search 和 Pareto archive，不能夸大为“深度强化学习主导性能提升”。第二，RevKit 和 ROS 的比较仍包含 proxy 或 adapter 成分，后续若要投稿到更严格的量子编译会议，应继续复现官方 ROS 或 legacy RevKit/CirKit 完整流程。第三，phase/Rz 分支虽然已有 Affine-FPRM 搜索，但仍停留在 parity-gadget 级 proxy；下一步应加入 learned phase/Rz policy、旋转谱优化和实际近似旋转序列审计。第四，所有结果都是逻辑层资源，不包含 routing、native gate mapping 或设备噪声。

因此，本文当前最稳妥的投稿定位是：一种逻辑层、资源约束、可验证的神经 MCTS Boolean

oracle synthesis 方法。它可以声称在 T-count 和 weighted score 上显著优于多类 baseline，并在高维项集上有强符号验证和小样本完整 bridge；它不能声称硬件级最优、CNOT-only 支配 SSHR、或 phase/Rz 全局最优。

8 结论

本文提出 Resource-NMCTS，将量子布尔函数 oracle 综合建模为 ANF/FPRM 项集合上的资源约束搜索问题，并结合神经动作先验、MCTS、布尔环线性因子筛选、frontier policy、Pareto archive 和 guard 生成逻辑层可逆线路。实验显示，Pareto-Resource-NMCTS 在传统函数上相对 ANF、ESOP、SSHR-H 和 ESOP-MILP 获得稳定低 T-count 与低 score 优势，在 ROS-style LUT proxy 和 official-header mockturtle XAG probe 下仍保持优势。内部消融说明主要增益来自 affine search、资源感知 search 和 Pareto archive，learned prior 是小幅正向模块。Affine-FPRM phase search 进一步把 RevKit/Rz 边界推进到可验证的 phase oracle 搜索分支。

当前 v36 已经从实验日志推进为图表化投稿主稿，但目标尚未完全结束：更强的 learned phase/Rz-aware search、官方 ROS 或 legacy RevKit/CirKit 完整复现，以及面向目标期刊模板的英文投稿稿仍是下一阶段必须完成的工作。

参考文献

- [1] R. Wille and R. Drechsler. BDD-based synthesis of reversible logic for large functions. In *Proceedings of DAC*, pp. 270–275, 2009.
- [2] G. Meuli, M. Soeken, M. Roetteler, and G. De Micheli. ROS: Resource-constrained oracle synthesis for quantum computers. *Electronic Proceedings in Theoretical Computer Science*, 318:119–130, 2020.
- [3] G. Meuli, M. Soeken, and G. De Micheli. Xor-And-Inverter Graphs for quantum compilation. *npj Quantum Information*, 8:7, 2022.
- [4] M. Soeken, G. Meuli, E. Testa, H. Nada, M. Reymond, B. Dang, G. De Micheli, and A. Mishchenko. The EPFL logic synthesis libraries. *arXiv:1805.05121*, 2018.
- [5] M. Soeken and collaborators. RevKit: Python quantum compilation library. <https://github.com/msoeken/revkit>.
- [6] M. Soeken and collaborators. mockturtle: logic network optimization and synthesis. <https://mockturtle.readthedocs.io/>.
- [7] Q. Zheng, Y. Xu, J. Zhang, Z. Su, and S. Zheng. CNOT oriented synthesis for small-scale Boolean functions using spatial structures of parallelotopes. In *Proceedings of ICCAD*, 2025.
- [8] D. Tsaras, A. Grosnit, L. Chen, Z. Xie, H. Bou-Ammar, and M. Yuan. ShortCircuit: AlphaZero-driven circuit design. *arXiv:2408.09858*, 2024.

- [9] S. Rietsch, A. Y. Dubey, C. Ufrecht, M. Periyasamy, A. Plinge, C. Mutschler, and D. D. Scherer. Unitary synthesis of Clifford+T circuits with reinforcement learning. In *IEEE International Conference on Quantum Computing and Engineering*, pp. 824–835, 2024.
- [10] J. Riu, J. Nogu  , G. Vilaplana, A. Garcia-Saez, and M. P. Estarellas. Reinforcement learning based quantum circuit optimization via ZX-calculus. *Quantum*, 9:1758, 2025.
- [11] M. Machiya, M. Menickelly, P. Hovland, and J. Liu. MonteQ: A Monte Carlo tree search based quantum circuit synthesis framework. *arXiv:2604.19029*, 2026.